

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание №3

Выполнил:

Майстренко Анастасия Николаевна

K3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать тестирование REST API сервиса аренды недвижимости средствами Postman и написать тесты внутри Postman. Результат — один комплексный сценарий по основному процессу приложения.

Эталонный сценарий из задания: регистрация - вход - список объявлений - фильтрация по списку - просмотр случайного объявления из списка - старт переписки по объявлению.

Ход работы

1. Объект и инструмент тестирования

Объектом тестирования выступает REST API, реализованное в LP1 (Node.js + Express + TypeORM). Тестирование выполнено в Postman: создана коллекция, в которой каждый запрос содержит тесты на языке JavaScript (объект `pm`). Базовый адрес API задан переменной коллекции `baseUrl`.

Коллекция приложена к работе в файле *rental-api-tests.postman_collection.json* и импортируется в Postman через File - Import.

2. Структура коллекции и переменные

Коллекция самодостаточна: чтобы основной сценарий выполнялся на чистой базе данных, в начало вынесена папка «00 — Подготовка данных», которая создаёт арендодателя, удобство и объявление. Далее идёт основной сценарий из семи шагов.

Передача данных между запросами реализована через переменные коллекции (chaining). В тестах одного запроса значения сохраняются методом `pm.collectionVariables.set`, а в следующих запросах подставляются синтаксисом `{{...}}`. Используемые переменные:

- `accessToken` — JWT-токен, полученный при входе, подставляется в заголовок `Authorization`;
- `tenantEmail` — e-mail пользователя, генерируется в pre-request скрипте, чтобы повторные запуски не конфликтовали;
- `filteredIds` — массив `id` объявлений после фильтрации;
- `propertyId` — `id` случайно выбранного объявления;
- `ownerId` — `id` владельца объявления (адресат переписки);
- `conversationId` — `id` созданного диалога.

3. Описание комплексного сценария

Основной процесс воспроизводит путь пользователя от регистрации до начала общения по объявлению. Шаги сценария и проверки приведены в таблице 1.

Таблица 1 — Шаги сценария и проверки

№	Запрос	Проверки (pm.test)
1	POST /auth/register	статус 201; есть access/refresh токены; пароль не возвращается; время ответа < 1 с
2	POST /auth/login	статус 200; получен accessToken; email совпадает с зарегистрированным
3	GET /properties	статус 200; структура { items, meta }; список не пуст; у объявления есть ключевые поля
4	GET /properties?type=studio&priceMax=3000	статус 200; есть результаты; все элементы соответствуют фильтру по типу и цене
5	GET /properties/{id}	статус 200; id совпадает; есть фото, удобства, владелец; пароль владельца не раскрыт
6	POST /conversations	статус 201; создан диалог; привязан к объявлению; участник — владелец
7	POST /conversations/{id}/messages	статус 201; текст сохранён; сообщение непрочитано

Случайный выбор объявления (шаг 5) реализован в pre-request скрипте: из сохранённого массива filteredIds случайно выбирается id (Math.random) и записывается в переменную propertyId.

4. Пример теста внутри Postman

Тесты пишутся во вкладке Scripts → Post-response (ранее Tests). Пример проверки шага фильтрации:

```
pm.test('Статус 200 ОК', () => {
  pm.response.to.have.status(200);
  const json = pm.response.json();
  pm.test('Все результаты соответствуют фильтру', () => {
    json.items.forEach((p) => {
      pm.expect(p.propertyType).to.eql('studio');
      pm.expect(p.pricePerDay).to.be.at.most(3000);
    });
  });
});
```

5. Запуск и результаты

Сценарий запускался против работающего API (через Postman Collection Runner; для автоматической проверки также использовался Newman — консольный запускатель Postman-коллекций). Итоговый результат прогона:

- выполнено запросов: 10;
- выполнено тест-скриптов: 10; pre-request скриптов: 3;

- пройдено проверок (assertions): 32 из 32; провалено: 0;
- среднее время ответа: 17 мс; общая длительность прогона: 361 мс.

Все проверки пройдены успешно, что подтверждает корректную работу основного процесса приложения end-to-end.

Вывод

В ходе работы средствами Postman реализован комплексный сценарий тестирования основного процесса сервиса аренды недвижимости: регистрация, вход, получение и фильтрация списка объявлений, просмотр случайного объявления и старт переписки по нему. Для каждого запроса написаны тесты на проверку статус-кодов, структуры и содержимого ответов, а также бизнес-правил (соответствие фильтру, отсутствие пароля в ответе, привязка диалога к объявлению).

Передача данных между запросами организована через переменные коллекции, что делает сценарий цельным и воспроизводимым. Прогон коллекции (32 проверки) завершился без ошибок, что подтверждает работоспособность API.